DYNATRACE KNOWLEDGE TRANSFER

From Code to Root Cause

How Samsung SmartStore is observed end-to-end on AWS — OneAgent, Davis AI, Dashboards, and SLOs.

Session length	30–60 minutes
Audience	SRE / Cloud / DevOps team, interview panel
Format	Story-driven walkthrough with a live demo
Application	Samsung SmartStore (Flask, Gunicorn, Nginx, Docker, EC2)

How to Use This Guide

This handbook is the speaking script and reference document for the Dynatrace KT session. It follows one continuous story — a single user request travelling through Samsung SmartStore — rather than a feature-by-feature tour. That thread naturally introduces architecture, observability concepts, OneAgent, dashboards, SLOs, and root-cause analysis in the order a real incident would surface them.

Read it top to bottom as a script, or jump to any Part as a standalone reference. Each Part lists an approximate speaking time so the whole session lands inside a 30–60 minute slot.

Part	Topic
1	Introduction & the 2 AM outage
2	Application architecture
3	Monitoring vs. Observability
4	Dynatrace architecture (OneAgent → Davis AI)
5	End-to-end data flow (master narrative)
6	Live demo — setup, infra, hosts, processes
7	SLI / SLO / error budget
8	Davis AI & production example
9	Kubernetes & future roadmap

Part 1 — Introduction: The 2 AM Outage

> *"It's 2 AM. Our e-commerce website is down. Customers can't place orders. As an SRE, how do we quickly find the root cause?"*

That question is the reason this session exists. By the end, the audience should understand not just what Dynatrace is, but how it answers that exact question in minutes instead of hours.

Open with the problem, not a definition. Then set the agenda in one breath:

- Explain how Dynatrace works, in plain language.
- Walk through the Samsung SmartStore architecture.
- Demonstrate the live application and its telemetry.
- Generate a failure on purpose.

- Show how Dynatrace — and Davis AI — detect and explain it automatically.

This hooks the audience immediately and gives them a reason to keep listening through the architecture and theory sections that follow.

Part 2 — Application Architecture

Before talking about Dynatrace at all, the audience needs to know what is actually being monitored. Draw or show this stack:

Layer	Role
Route53 DNS	Resolves the domain to the EC2 instance
Nginx	Reverse proxy; serves static files directly
Gunicorn	WSGI application server running Flask workers
Flask — Samsung SmartStore	Application logic: catalog, cart, checkout, orders
SQLite	Database for products, users, and orders
Docker container	Packages the app and its dependencies
AWS EC2 (Amazon Linux 2023)	The host all of the above runs on

Explain it as a request's journey: a user hits the site, DNS resolves it, Nginx receives the request, Gunicorn hands it to Flask, Flask talks to SQLite, and the whole thing runs inside a Docker container on EC2. Once the audience has this mental model, everything Dynatrace shows later has a place to "language."

Part 3 — Monitoring vs. Observability

This is the most important concept in the whole session, and it's worth slowing down for. Many people treat "monitoring" and "observability" as synonyms. They aren't.

Monitoring

Monitoring means continuously checking system health against thresholds — CPU > 80%, memory > 90%, disk > 85%. When a threshold is crossed, an alert fires. Monitoring tells you something is wrong. It doesn't tell you why.

Observability

Observability combines three signal types — metrics, logs, and traces — to explain why. Instead of "CPU is high," observability tells you: the Gunicorn worker is waiting on a database query, that query is scanning a table without an index, and it's taking eight seconds.

A Worked Example

> *"Users report that login is slow. Monitoring shows CPU at 40%, memory at 50%, disk normal — everything looks fine. Observability shows the Authentication Service waiting six seconds on the database. Now you know exactly where to look."*

The Three Pillars

- Metrics — numerical values over time: CPU, memory, response time, request count, error rate. Metrics tell you what is happening.
- Logs — detailed records: "Database connection failed," exceptions, timeouts. Logs tell you exactly what happened.

- Traces — the path of one request end to end: Load Balancer → App → Auth → Database. Traces tell you which specific hop is slow.

Part 4 — Dynatrace Architecture

With the concepts in place, introduce the actual components.

Component	What it does
OneAgent	Installed on every host; auto-discovers processes and collects metrics, logs, and traces — no code changes required
ActiveGate (optional)	A secure gateway used in enterprise or private-network environments to route telemetry
Dynatrace SaaS	Stores and processes all incoming telemetry
Davis AI	Correlates everything automatically and identifies the root cause

The flow is linear and easy to remember: Users → Application → OneAgent → ActiveGate (optional) → Dynatrace SaaS → Davis AI → Dashboards, Problems, and Alerts.

Part 5 — The End-to-End Data Flow (Master Narrative)

This is the single thread that ties every earlier concept together. Say it once, cleanly, and the whole session clicks into place:

> *"A user opens the website. The request reaches EC2, inside the Docker container. Flask processes it and queries SQLite. Simultaneously, OneAgent captures metrics, logs, and traces from every layer. That data is sent to Dynatrace SaaS. Davis AI correlates the telemetry. Dashboards update. If something is wrong, a Problem is detected, the relevant SLO is evaluated, and an alert is generated. The engineer opens the trace, finds the root cause, and fixes the issue — all with a much lower mean time to resolution than manual investigation."*

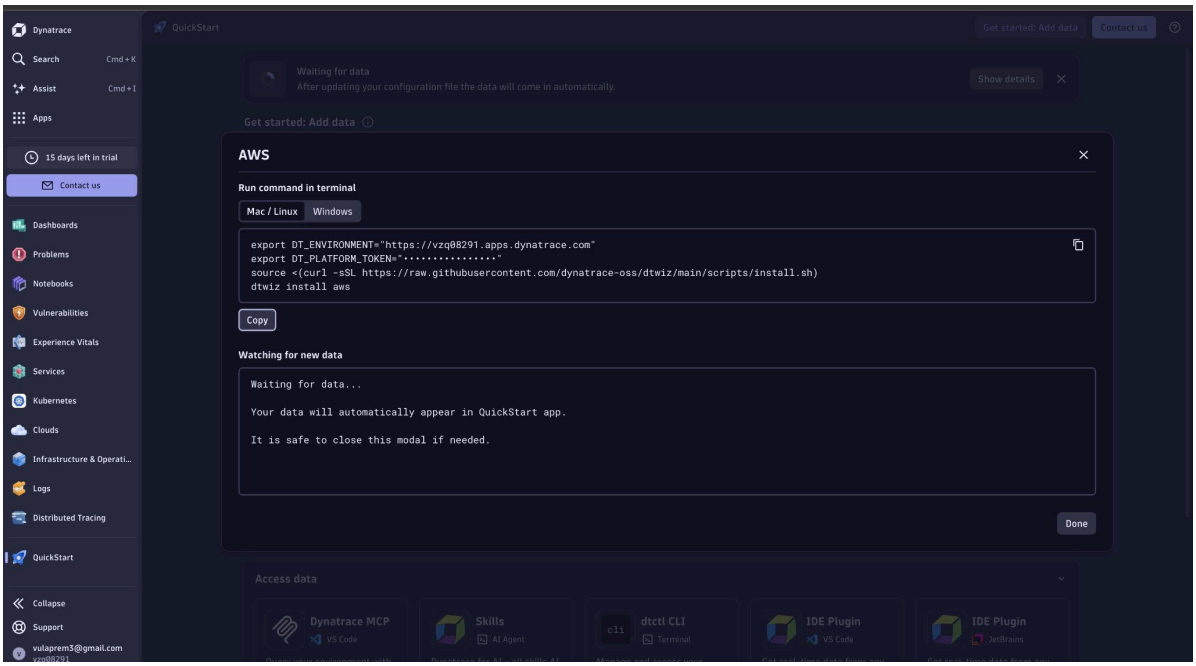
Everything from here forward in the live demo is simply zooming into one part of that sentence.

Part 6 — Live Demo Walkthrough

Six screenshots from the actual Samsung SmartStore + Dynatrace + AWS setup, in the order they'd naturally come up during a live demo.

6.1 — OneAgent & AWS Integration Setup

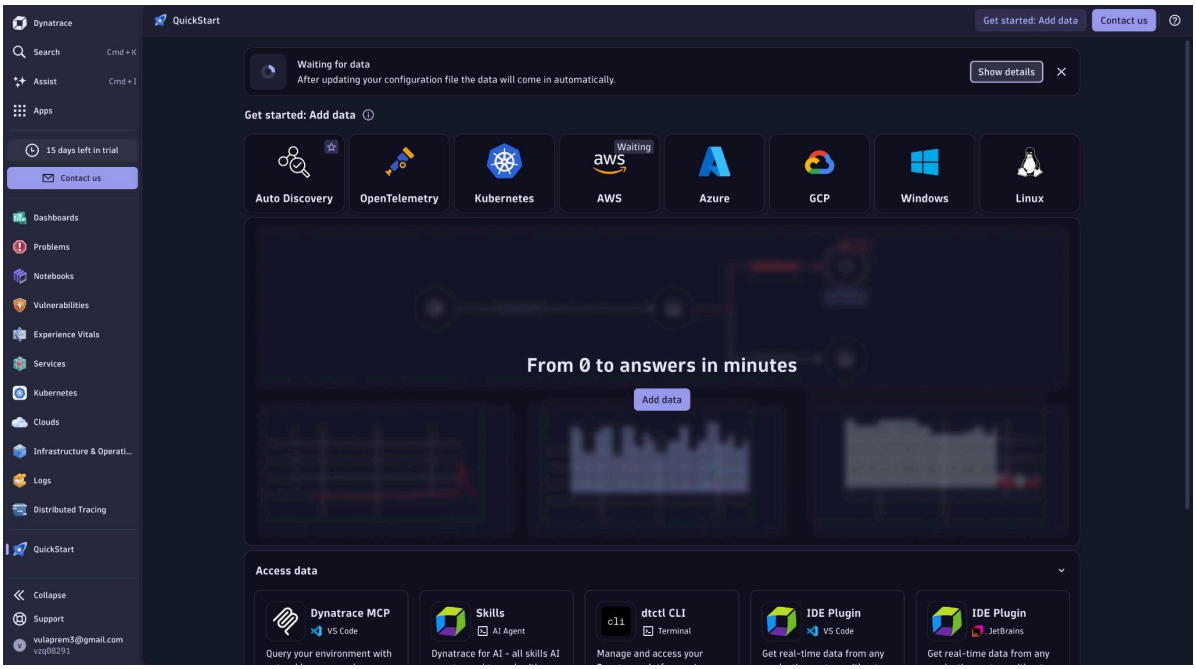
We connect Dynatrace to the AWS account and install OneAgent on the EC2 host using the dtwiz installer. It detects the platform automatically and pulls the correct OneAgent build — no manual instrumentation of the Flask code is required.



dtwiz AWS onboarding — OneAgent install command generated for the target host.

6.2 — Infrastructure as Code Deployment

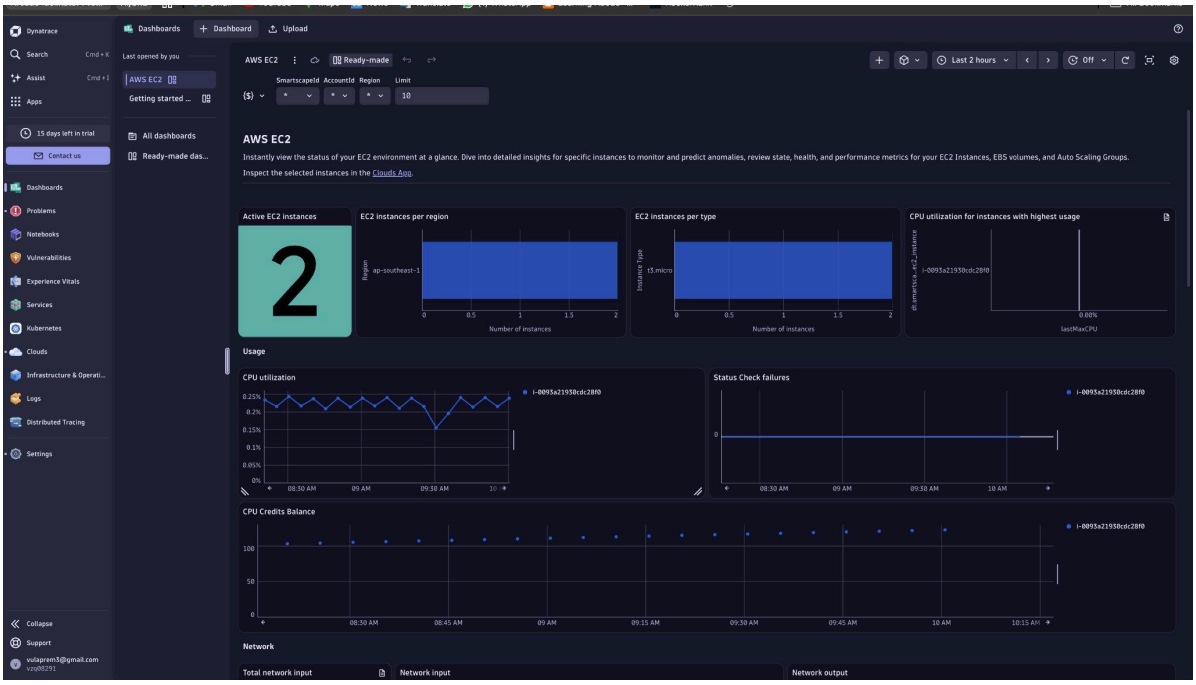
The AWS integration itself is deployed via a CloudFormation stack — the same infrastructure-as-code philosophy the GitHub Actions pipeline uses for the application. It provisions the IAM roles and monitoring configuration Dynatrace needs to read AWS metrics.



CloudFormation stack deploying the Dynatrace – AWS integration.

6.3 — Infrastructure Monitoring (AWS EC2)

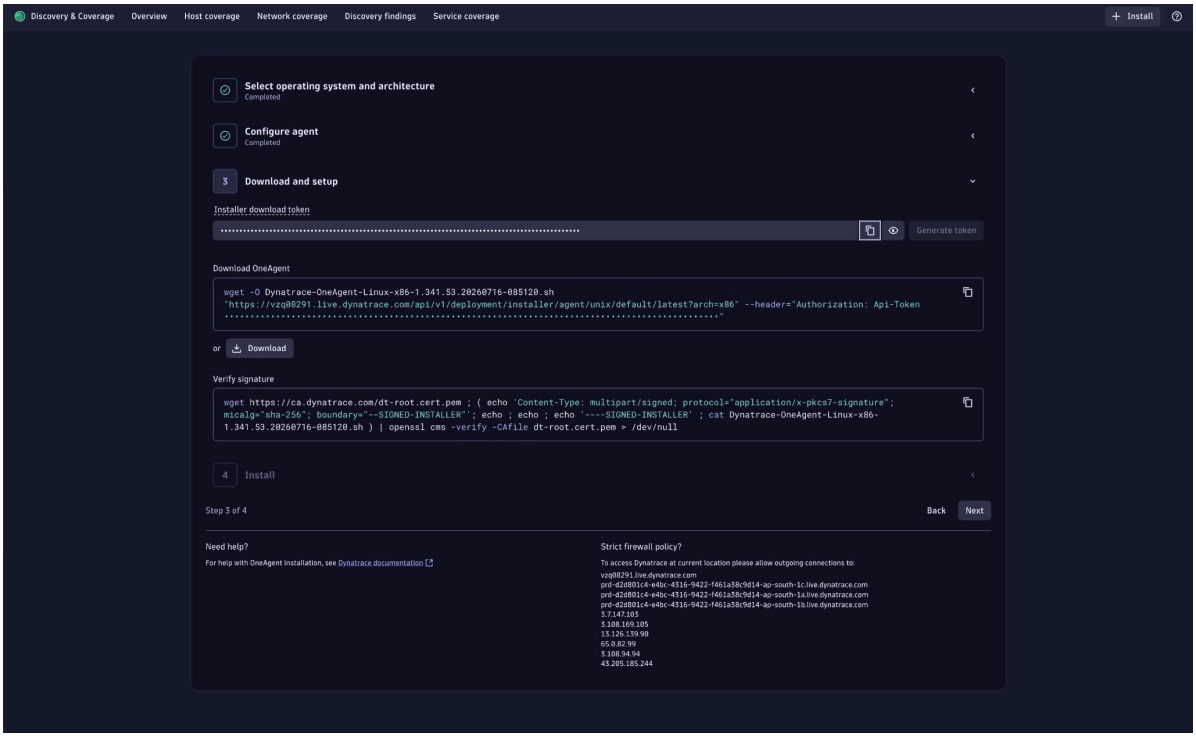
A ready-made AWS EC2 dashboard shows instance count, type, and region, plus CPU utilization and status-check history per instance — out of the box, with zero custom configuration. This is pure monitoring: healthy numbers here don't guarantee a healthy application, which is exactly the point made earlier about monitoring vs. observability.



Ready-made AWS EC2 dashboard.

6.4 — Host Health Deep Dive

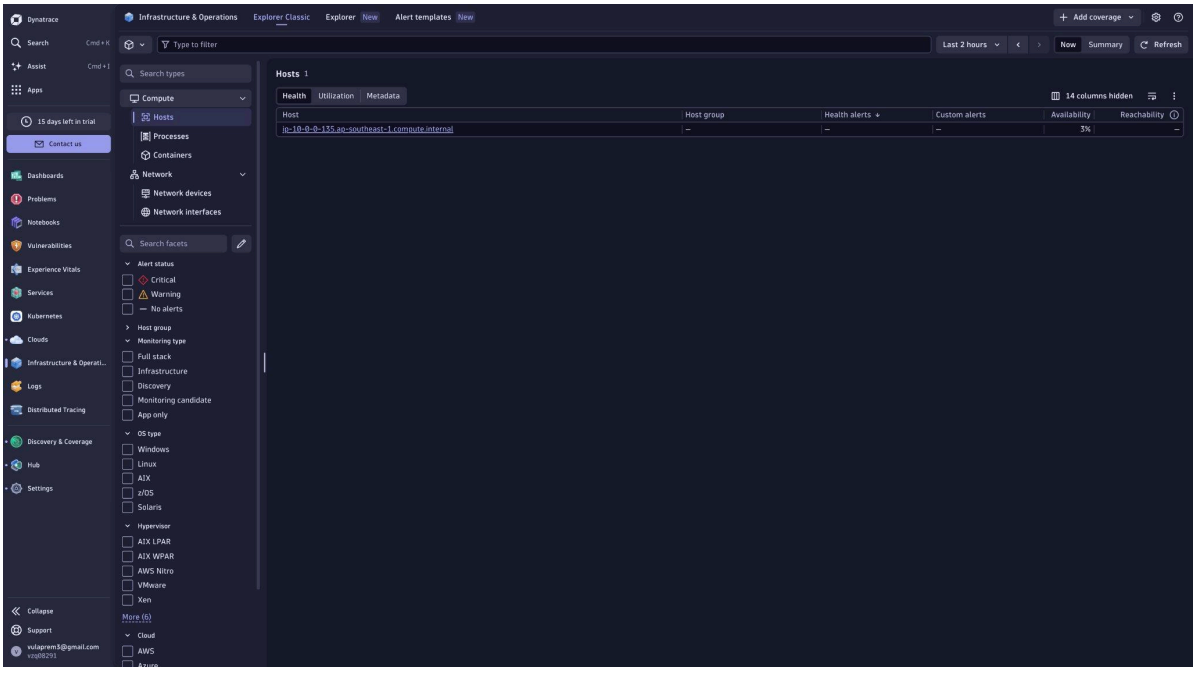
Drilling into the host itself: CPU, memory, disk, and network are all visible, and availability is tracked continuously — this feeds directly into the uptime SLO discussed in Part 7. OneAgent auto-discovers every process on the host without extra configuration.



Host overview — CPU, memory, disk, and network for the EC2 instance.

6.5 — Process & Container-Level Monitoring

OneAgent breaks CPU and memory down per process, not just per host, so the Samsung SmartStore Docker container running Gunicorn is visible directly. Confirming the /health endpoint from the terminal shows both the app and the database are reachable. This granularity is what turns “the host is fine” into “this exact process is the problem.”

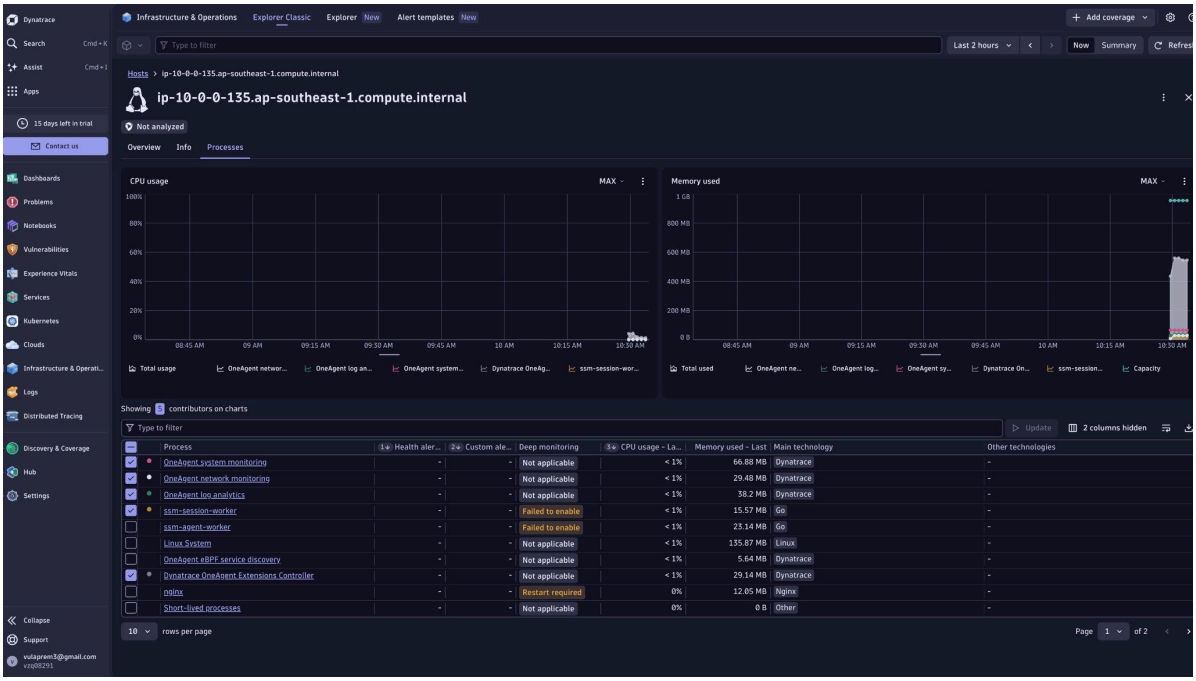


Process list and container view, plus a terminal health check against the running container.

Part 7 — SLI, SLO, and Error Budget

- SLI (Service Level Indicator) — the actual measurement, e.g. real response time.
- SLO (Service Level Objective) — the business target, e.g. 99% of requests under 500ms.
- Error Budget — how much failure is tolerated before the objective is breached.

Samsung SmartStore's response-time SLO is currently sitting at 100% status against a 99% target, with a 1% error budget over the last 24 hours.



Samsung SmartStore — Response Time SLO.

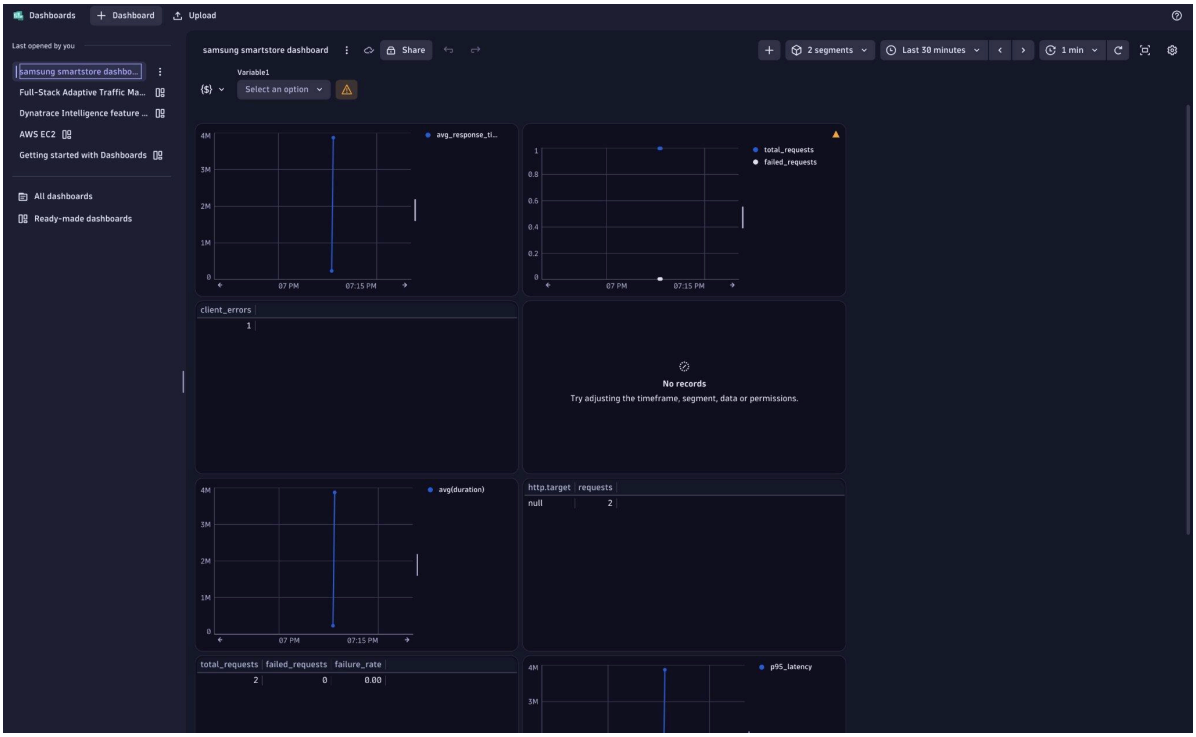
Distributed tracing is what makes SLOs actionable rather than just a number on a dashboard. Dynatrace's Adaptive Traffic Management captures a configurable volume of full-stack traces automatically, keeping trace volume within the host's memory budget — an important constraint on a t3.micro instance. Every captured trace can be replayed request-by-request across services, which is what lets Davis AI point to the exact slow hop rather than just the slow host.

```
[root@ip-10-0-0-135 samsung-smartstores]# ls -l /opt/dynatrace/oneagent/agent
total 508
drwxr-xr-x. 2 root root      163 Jul 16 08:37 SELinuxPolicy
-rw-r--r--. 1 root dtuser 153889 Jul 16 08:35 THIRDPARTYLICENSEREADME.txt
-rw-r--r--. 1 root dtuser 241816 Jul 16 08:35 agentinstaller-sbom-external.cdx.json
drwxr-xr-x. 3 root dtuser     24 Aug  2 04:58 authorizedkeys
drwxr-xr-x. 3 root dtuser     53 Aug  2 04:58 bin
drwxr-xr-x. 2 root dtuser  16384 Aug  2 04:58 conf
drwxr-xr-x. 6 root dtuser     74 Aug  2 04:58 datasources
-rw-rw-r--. 1 root dtuser      0 Aug  2 04:58 dt_fips_disabled.flag
drwxr-xr-x. 2 root dtuser     22 Aug  2 04:58 initscripts
-rw-r--r--. 1 root dtuser     25 Jul 16 08:34 installer.version
drwxr-xr-x. 2 root dtuser  16384 Jul 16 08:37 lib
drwxr-xr-x. 2 root dtuser  16384 Aug  2 04:58 lib64
drwxr-xr-x. 2 root dtuser     81 Jul 16 08:37 libmusl64
lrwxrwxrwx. 1 root dtuser     52 Aug  2 04:58 rdp -> /opt/dynatrace/oneagent/agent/lib64/oneagentdumpproc
drwxr-xr-x. 4 root dtuser    121 Aug  2 04:58 res
drwxr-xr-x. 4 root dtuser     74 Aug  2 04:58 tools
-rwxr-x---. 1 root dtuser  62488 Jul 16 08:35 uninstall.sh
[root@ip-10-0-0-135 samsung-smartstores]# docker run -d \
  --name samsung-smartstore \
  --restart unless-stopped \
  -p 5000:5000 \
  -e FLASK_ENV=production \
  -e SECRET_KEY=SamsungSecretKey \
  -v /opt/samsung-smartstores/instance:/app/instance \
  -v /opt/dynatrace/oneagent:/opt/dynatrace/oneagent:ro \
  -e LD_PRELOAD=/opt/dynatrace/oneagent/agent/lib64/liboneagentproc.so \
  samsung-smartstore
6a5a1de854cece56b02eb7c26e8d32b2f7dc64a88d4cb959fc83934f517b9b87
[root@ip-10-0-0-135 samsung-smartstores]# docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED         STATUS         PORTS                                       NAMES
6a5a1de854ce   samsung-smartstore   "gunicorn --bind 0.0…"   7 seconds ago   Up 6 seconds   0.0.0.0:5000->5000/tcp, :::5000->5000/tcp   samsung-smartstore
[root@ip-10-0-0-135 samsung-smartstores]# curl http://127.0.0.1:5000/health
{"database":"UP","service":"samsung-smartstore","status":"UP"}
[root@ip-10-0-0-135 samsung-smartstores]# docker exec -it samsung-smartstore bash
ERROR: ld.so: object '/opt/dynatrace/oneagent/agent/lib64/liboneagentproc.so' from LD_PRELOAD cannot be preloaded (cannot open shared object file): ignored.
root@6a5a1de854ce:/app# echo $LD_PRELOAD
/opt/dynatrace/oneagent/agent/lib64/liboneagentproc.so
root@6a5a1de854ce:/app# ls /opt/dynatrace/oneagent/agent/lib64/
ERROR: ld.so: object '/opt/dynatrace/oneagent/agent/lib64/liboneagentproc.so' from LD_PRELOAD cannot be preloaded (cannot open shared object file): ignored.
fips.so   fips3.so       libelf.so.1          nettracer-bpf.o    oneagentdynamizer     oneagentextensions     oneagentloganalytics  oneagentnetwork    oneagentpreinjectcheck
fips1.so  fips4.so       liboneagentaudit.so  oneagentdmidecode  oneagentebpfdiscovery  oneagenthelper         oneagentmntconstat    oneagentos         oneagentruncwrapper
fips2.so  fipsmodule.cnf  liboneagentdynamizer.so  oneagentdumpproc   oneagenteventstracer   oneagentinstallaction  oneagentnettracer     oneagentosconfig   oneagentwatchdog
root@6a5a1de854ce:/app#
```

Full-Stack Adaptive Traffic Management and trace capture.

7.1 — The Custom Application Dashboard

Purpose-built for Samsung SmartStore: average response time, total vs. failed requests, and client errors, with failure rate computed live as 5xx ÷ total requests — the headline reliability number. Latency and traffic panels sit side by side so a spike in one correlates instantly with the other. This is the dashboard the team actually watches during a release.



Samsung SmartStore custom dashboard — response time, request volume, and error rate.

Part 8 — Davis AI & a Production Example

Davis AI's value is that it doesn't stop at “CPU is high.” It explains which host, which process, which service, which dependency, and the root cause — instead of leaving the engineer to investigate manually.

Walkthrough: “Checkout Is Slow”

Step	What Happens
1	Users complain that checkout is slow
2	Dynatrace opens a Problem
3	Response time is flagged as high
4	The engineer opens the distributed trace
5	The trace shows the database query as the slow hop
6	Root cause is found and the issue is fixed

This mirrors the real production troubleshooting pattern SmartStore's SRE team would use — and it's the same six-step shape as the checkout-latency example used earlier for Kubernetes workloads.